

KnowledgeBase — Technical Deep Dive

A component-by-component breakdown of how the system works under the hood.

Table of Contents

1. [System Overview](#)
 2. [Data Model](#)
 3. [Configuration Layer](#)
 4. [Ingestion Pipeline](#)
 5. [Embedding Layer](#)
 6. [Semantic Search](#)
 7. [RAG Chat Pipeline](#)
 8. [LLM Provider Abstraction](#)
 9. [Meeting Intelligence Pipeline](#)
 10. [Time Task Tracker Integration](#)
 11. [TTT Query Layer \(chat\)](#)
 12. [Frontend Architecture](#)
 13. [Deployment Model](#)
-

1. System Overview

KnowledgeBase is a full-stack RAG (Retrieval-Augmented Generation) application. Its core loop is:

File / transcript

- extract text
- chunk
- embed (768-dim vector)
- store in pgvector

User question

- embed question
- cosine similarity search → top-K chunks
- assemble prompt (system + context + history + question)
- LLM inference
- answer + source citations

On top of this core loop sit two additional capabilities:

- **Meeting intelligence** — transcripts are ingested like any document, then a structured summary is generated and written to the Time Task Tracker database.
- **TTT chat context** — when a user asks about time entries, hours, or billing, the RAG pipeline fetches rows from the TTT `time_entries` table and injects them as structured context alongside vector chunks.

Service topology

Service	Runtime	Port	Role
PostgreSQL + pgvector	Docker / Neon (cloud)	5433 local / managed	Stores document chunks, 768-dim vectors, metadata
Ollama	Native (local)	11434	Local embedding (<code>nomic-embed-text</code>) + local LLM (<code>llama3.2</code>)
FastAPI / uvicorn	Python	8000	REST API — all backend logic
Vite / React	Node	5173	Frontend SPA
TTT PostgreSQL	Neon (cloud)	managed	Separate DB for <code>time_entries</code>

2. Data Model

`documents` table (pgvector DB)

Defined in [backend/kb/db.py](#).

Column	Type	Description
<code>id</code>	<code>INTEGER</code> PK	Auto-increment
<code>source</code>	<code>VARCHAR(1024)</code>	Original filename or file path — the dedup key
<code>file_type</code>	<code>VARCHAR(64)</code>	<code>pdf</code> , <code>text</code> , <code>docx</code> , <code>image</code> , <code>generic</code>
<code>chunk_index</code>	<code>INTEGER</code>	Position of this chunk within the source (0-based)
<code>content</code>	<code>TEXT</code>	Raw text of the chunk
<code>embedding</code>	<code>Vector(768)</code>	pgvector column — cosine similarity target
<code>created_at</code>	<code>TIMESTAMP</code>	Insert time

Column	Type	Description
doc_metadata	JSONB	Structured fields extracted from meeting headers: meeting_date, meeting_title, organizer, attendees, platform, meeting_time, project_code, billable, duration_minutes

The `doc_metadata` column is added via `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` in `init_db()` so older deployments without it are migrated transparently on startup.

An index on `source` makes the dedup check and `source_filter` queries fast.

time_entries table (TTT DB)

Lives in a separate Neon database. Written to by [backend/kb/pusher.py](#), read by [backend/kb/ttt.py](#).

Column	Type	Description
id	TEXT PK	UUID string
project_code	TEXT	Parsed from meeting header <code>Project:</code> field
task_type	TEXT	Always <code>meeting</code> for KB-pushed entries
duration_minutes	NUMERIC	Parsed from <code>Meeting Duration:</code> header
entry_date	DATE	Parsed from <code>Date:</code> header
start_time	TIMESTAMPTZ	Parsed from <code>Time:</code> header range (naive, no tz conversion)
end_time	TIMESTAMPTZ	Parsed from <code>Time:</code> header range (naive, no tz conversion)
description	TEXT	LLM-generated meeting summary
meeting_title	TEXT	Parsed from <code>Meeting Title:</code> header
billable	BOOLEAN	Parsed from <code>Billable: Yes/No</code> header
confidence	NUMERIC	Fixed at <code>0.75</code> for KB-pushed entries
status	TEXT	Fixed at <code>logged</code>
organizer	TEXT	Parsed from <code>Organizer:</code> header

Column	Type	Description
attendees	TEXT	Parsed from Attendees: header

3. Configuration Layer

File: [backend/kb/config.py](#)

All configuration is environment-driven. `config.py` calls `load_dotenv()` against up to three candidate paths in priority order:

1. `KB_ENV_FILE` env var (explicit override)
2. `./env` in the current working directory
3. `.env` in the project root

Every other module imports constants directly from `config.py` — nothing reads `os.environ` elsewhere.

4. Ingestion Pipeline

Files: [backend/kb/ingest.py](#) · [backend/kb/extractors.py](#)

Entry points

- **API:** `POST /upload` (multipart file) and `POST /ingest-meeting` (multipart file)
- **CLI:** `kb ingest <path>`

Both paths converge on `ingest(path, force=False, source_name=None)`.

Step 1 — File discovery

`_iter_files(root)` walks the path. If it is a single file it yields it directly. If it is a directory it recursively yields all files whose suffix appears in `SUPPORTED_SUFFIXES`.

Step 2 — Dedup check

Before doing any work, `_already_indexed(session, source_key)` checks for an existing row with the same `source`. If found and `force=False`, the file is skipped. If `force=True`, all existing rows for that source are deleted before re-ingesting.

The `source_key` is the caller-supplied `source_name` (the original upload filename) when ingesting a single file, so the DB key is always the human-readable name rather than a temp path.

Step 3 — Extraction and chunking

`extract(path)` in `extractors.py` dispatches by file extension:

Suffix	Extractor	Notes
<code>.pdf</code>	<code>extract_pdf</code>	pypdf — concatenates all page text
<code>.docx</code> , <code>.doc</code>	<code>extract_docx</code>	python-docx — joins paragraph text
<code>.png</code> , <code>.jpg</code> , <code>.jpeg</code> , <code>.gif</code> , <code>.bmp</code> , <code>.tiff</code> , <code>.webp</code>	<code>extract_image</code>	Pillow + pytesseract OCR
<code>.txt</code> , <code>.md</code> , <code>.csv</code> , <code>.json</code> , <code>.yaml</code> , <code>.html</code> , <code>.py</code> , <code>.js</code> , <code>.ts</code> , <code>.go</code> , etc.	<code>extract_txt</code>	Plain read + meeting metadata extraction
anything else	<code>extract_generic</code>	Strips non-ASCII bytes, extracts printable characters

All extractors funnel through `_chunk_text(text, size=800, overlap=100)` which produces fixed-size character chunks with 100-character overlap between consecutive chunks to preserve context at boundaries.

Meeting metadata extraction — `extract_txt` additionally calls

`extract_meeting_metadata(raw)` which scans the text for structured header fields using regex patterns:

Pattern	Stored as
Date: ...	meeting_date (normalised to ISO-8601)
Meeting Title: ...	meeting_title
Time: ...	meeting_time (raw string, e.g. 2:00 PM – 2:45 PM CST)
Organizer: ...	organizer
Attendees: ...	attendees
Platform: ...	platform
Project: ...	project_code
Billable: ...	billable (normalised to bool)

Pattern	Stored as
Meeting Duration: ...	duration_minutes (normalised to float)

Each chunk is prefixed with a short `[Meeting: ... | Date: ... | Organizer: ...]` header so every chunk carries context about its source meeting even when retrieved in isolation.

Step 5 — Embedding

For each chunk, `embed(chunk)` is called synchronously. The result is a `list[float]` of length `EMBED_DIMENSIONS` (768).

Step 6 — Database write

A `Document` ORM row is constructed and added to the session for each chunk. The session commits after all chunks of a single file are processed. A failure rolls back only that file.

5. Embedding Layer

File: [backend/kb/embedder.py](#)

The public interface is two functions:

```
embed(text: str) -> list[float]
embed_batch(texts: list[str]) -> list[list[float]]
```

Both delegate to the provider returned by `get_embedder()`, which reads `EMBED_PROVIDER` from config.

OllamaEmbedder

Calls Ollama's `/api/embed` endpoint via `httpx` with `Connection: close` headers to force HTTP/1.1 and avoid connection-reuse issues with Ollama's server. Returns `embeddings[0]` from the response.

NomicEmbedder

Calls the Nomic Atlas API at `https://api-atlas.nomic.ai/v1/embedding/text`. Uses `task_type: "search_document"` during ingestion. The same provider is used at query time — at query time `search_document` is still passed (the distinction between document and query task types is handled at the model level by Nomic).

Both providers output 768-dimensional vectors, matching the `Vector(768)` pgvector column.

6. Semantic Search

File: [backend/kb/search.py](#)

`search(query, top_k, file_type, source_filter)`

1. Calls `embed(query)` to get the query vector
2. Builds a SQLAlchemy query using pgvector's cosine distance operator `<=>`:

```
SELECT *, embedding <=> :query_vector AS distance
FROM documents
[WHERE file_type = :file_type]
[WHERE source ILIKE '%source_filter%']
ORDER BY distance
LIMIT :top_k
```

3. Converts distance to similarity score: `score = 1.0 - distance`
4. For each result, calls `_extract_snippet(content, query)` to produce a short excerpt centred on the best-matching region of the chunk

Snippet extraction

`_extract_snippet` uses a sliding window algorithm:

- Tokenises the query into non-trivial terms (≥ 3 chars, minus stop words)
- Finds all character positions where each term appears in the chunk
- Slides over match positions to find the window that covers the most distinct query terms
- Trims to sentence or word boundaries and adds ellipsis markers

Supporting queries

- `get_most_recent_meeting_date()` — queries `doc_metadata->>'meeting_date'` across all documents to find the most recent meeting date. Used by the temporal meeting re-ranking logic in chat.
 - `list_sources()` — `GROUP BY source, file_type` with chunk count
 - `delete_source(source)` — deletes all rows matching `source` exactly
-

7. RAG Chat Pipeline

File: [backend/kb/chat.py](#)

Entry point: `ask(question, history, top_k, source_filter, file_type, skip_ttt)`

Step 1 — Intent classification (regex, pre-LLM)

Two classifiers run on the raw question text before any retrieval:

`_is_temporal_meeting_query(question)` — matches patterns like "last meeting", "most recent standup", "latest sync". When true, triggers meeting date re-ranking and forces TTT meeting history lookup.

`is_ttt_query(question)` (from `ttt.py`) — matches patterns like "hours logged", "billable", "time entries", "what did I work on". When true, fetches structured TTT rows.

Step 2 — Vector retrieval

Calls `search(question, top_k, file_type, source_filter)`. Returns up to `top_k` `SearchResult` objects ordered by cosine similarity.

Step 3 — Temporal meeting re-ranking

If `_is_temporal_meeting_query` was true:

1. `get_most_recent_meeting_date()` fetches the latest `meeting_date` in `doc_metadata`
2. Retrieved chunks are partitioned: chunks from that date first, all others after
3. If no chunks from the most recent date were in the top-K, a second search is run with `top_k * 2` and the same partitioning is applied

Step 4 — TTT context injection

If `skip_ttt=False` (default) and either temporal or TTT intent was detected, `query_ttt(question)` fetches structured rows from the TTT database and formats them as a text block. This block is prepended to the vector context — structured data before unstructured prose.

`skip_ttt=True` is passed from the `/summarize-meeting` endpoint to prevent historical TTT entries from contaminating the summary of the specific meeting being processed.

Step 5 — Prompt construction


```
[system]
You are a helpful assistant with access to a personal knowledge base and
Time Task Tracker (TTT) database of logged work entries.
Answer the user's question using ONLY the context passages provided below
...
Context:
{ttt_rows}

[1] filename.txt (chunk 2, date 2026-08-16, score 0.821):
<chunk text>

[2] ...

[user turn N-2] ...
[assistant turn N-1] ...
[user] <current question>
```

The conversation history is appended as alternating `user / assistant` messages so the LLM has multi-turn context. The KB retrieval always uses only the latest question (not the full history) so retrieval stays focused.

Step 6 — LLM call

`get_provider().chat(messages)` dispatches to the configured provider and returns a string. The system prompt uses a hard constraint — "Answer using ONLY the context passages" — with no fallback to model parametric knowledge.

Step 7 — Response

Returns `ChatResponse(answer: str, sources: list[dict])` where each source entry contains `source`, `score`, and `chunk_index`.

8. LLM Provider Abstraction

File: [backend/kb/llm.py](#)

All providers implement `BaseLLMProvider.chat(messages: list[dict]) -> str`. Messages follow the OpenAI format: `[{"role": "system"|"user"|"assistant", "content": "..."}]`.

OllamaProvider

Posts to `{OLLAMA_HOST}/api/chat` with `stream=False`. Returns `response["message"]` `["content"]`. Supports streaming via a generator (not wired to the API yet).

OpenAIProvider

Posts to `{OPENAI_BASE_URL}/chat/completions`. Works with any OpenAI-spec endpoint — OpenAI, Groq, GLM-4 (ZhipuAI), Anyscale, Together, etc. Returns `choices[0].message.content`.

WatsonxProvider

Two-step per call:

1. **IAM token exchange** — POST `https://iam.cloud.ibm.com/identity/token` with `grant_type=apikey` → `access_token`
2. **Inference** — POST `{WATSONX_URL}/ml/v1/text/chat?version=2023-05-29` with the token in the `Authorization` header

Parameters sent: `temperature=0`, `max_tokens=600`, `frequency_penalty=0`, `presence_penalty=0`, `top_p=1`. Temperature 0 is intentional — deterministic grounded answers for a RAG use case.

Note: The IAM token is fetched fresh on every call. There is no token cache. Each call adds ~200–400ms for the token exchange round-trip.

Adding a new provider

1. Subclass `BaseLLMProvider` in `llm.py` and implement `chat()`
2. Register it in `get_provider()` with a string key
3. Set `LLM_PROVIDER=<key>` in `.env`

9. Meeting Intelligence Pipeline

Files: [backend/api.py](#) · [backend/kb/extractors.py](#) · [backend/kb/pusher.py](#)

This pipeline is split across two HTTP requests to avoid Render's 30-second response timeout. The LLM summarization call alone can take 15–25 seconds, making a single-request design unreliable on hosted free-tier deployments.

Request 1 — POST /ingest-meeting

1. Saves the uploaded file to a temp path
2. Calls `ingest(tmp_path, force=force, source_name=file.filename)` — same pipeline as `/upload`
3. Deletes the temp file
4. Returns `{status: "ok", filename: "..."} immediately`

This step is fast (seconds). The file is now in the vector DB.

Request 2 — `POST /summarize-meeting`

Receives `{filename, project_code, organizer, attendees}` as JSON.

1. **Metadata lookup** — calls `kb_search(filename, top_k=1, source_filter=filename)` to retrieve the first chunk and extract `doc_metadata: date, title, time, duration, project, billable, organizer, attendees`
2. **RAG summarization** — calls `kb_ask(summary_question, source_filter=filename, top_k=3, skip_ttt=True)`. The `source_filter` scopes retrieval to only chunks from this file. `skip_ttt=True` prevents TTT history from contaminating the output.
3. **Time parsing** — `_parse_time_range(meeting_time, entry_date)` in `pusher.py` parses the raw time string (e.g. `2:00 PM – 2:45 PM CST`) into two naive `datetime` objects using regex. Times are stored without timezone conversion so the TTT displays wall-clock time as written.
4. **TTT push** — `push_meeting_entry(...)` inserts a row into `time_entries`. All fields are sourced from `doc_metadata` first, falling back to request body values, falling back to defaults. The `meeting_title` field uses the parsed `Meeting Title:` header, not the filename.
5. Returns the full `IngestMeetingResponse` including the LLM summary, source chunk citations, TTT entry ID, and any TTT push error.

Field sourcing priority

TTT field	Source
<code>meeting_title</code>	<code>doc_metadata.meeting_title</code> → filename
<code>entry_date</code>	<code>doc_metadata.meeting_date</code> → today
<code>start_time</code> / <code>end_time</code>	parsed from <code>doc_metadata.meeting_time</code> → null
<code>duration_minutes</code>	<code>doc_metadata.duration_minutes</code> → parsed from LLM summary → 60

TTT field	Source
project_code	request body → doc_metadata.project_code → parsed from LLM summary → filename stem
organizer	request body → doc_metadata.organizer → null
attendees	request body → doc_metadata.attendees → null
billable	doc_metadata.billable → false
description	LLM-generated summary

10. Time Task Tracker Integration

File: [backend/kb/pusher.py](#)

`push_meeting_entry()` uses a direct `psycopg2` connection to the TTT Neon database (bypassing any TTT API layer). The `INSERT ... ON CONFLICT (id) DO NOTHING` pattern means duplicate pushes are safe — the same UUID won't overwrite an existing entry.

Time range parsing

`_parse_time_range(time_str, entry_date)` handles formats like:

- `10:00 AM – 10:30 AM CST`
- `14:00 – 14:45`
- `2:00 PM – 2:45 PM`

The regex `(\d{1,2}:\d{2})\s*(AM|PM)?` finds all time tokens in the string. The first match becomes `start_time`, the second becomes `end_time`. Both are constructed as naive `datetime` objects (no `tzinfo`) so no UTC offset conversion is applied — the TTT displays them exactly as written.

Duration parsing

`_parse_duration_minutes(text)` applies two regexes to the LLM summary or the raw header value:

- `(\d+)\s*(?:hour|hr|h)\b` → multiply by 60
- `(\d+)\s*(?:minute|min|m)\b` → add directly

Defaults to 60 minutes if nothing is found.

11. TTT Query Layer (chat)

File: [backend/kb/ttt.py](#)

`query_ttt(question, limit, force_meetings)` is called from `chat.py` when TTT intent is detected. It classifies the question into one of four SQL shapes:

Shape	Trigger	Query
Meeting list	<code>force_meetings=True</code> or meeting history pattern	<code>SELECT ... WHERE task_type='meeting' ORDER BY entry_date DESC</code>
Aggregated totals	"total", "sum", "how many hours"	<code>SELECT SUM(duration_minutes), COUNT(*) GROUP BY project_code, task_type</code>
Billable filter	"billable"	<code>SELECT ... WHERE billable = TRUE</code>
Default recent	fallback	<code>SELECT ... ORDER BY entry_date DESC</code>

All shapes support:

- **Date range** — extracted from "today", "this week", "last month", etc. Defaults to ± 365 days from today (wide window to handle entries dated in the future, e.g. upcoming meetings)
- **Project filter** — `ILIKE '%project%'` extracted from "for Honda", "on Honda", "Honda meeting"
- **Count** — extracted from "last two meetings", "last 3 entries" — overrides the default `limit`

Results are formatted as a readable text block prepended with `[Time Task Tracker – N result(s), date to date]` and injected into the RAG prompt context ahead of the vector chunks.

12. Frontend Architecture

Files: [frontend/src/](#)

The frontend is a single-page React application built with Vite. All API communication goes through [frontend/src/api.js](#) which wraps `fetch()` calls and parses error responses from FastAPI's `{"detail": "..."} JSON shape.`

Components

Component	Responsibility
App.jsx	4-tab shell: Chat, Search, Upload, Meeting Upload. Tab state only — no global state management library.
Chat.jsx	Multi-turn conversation UI. Maintains history array locally. Sends full history with each request. Renders user/assistant bubbles, collapsible source citations, thinking state.
Search.jsx	Semantic search form with file type filter, source substring filter, top-K control. Renders snippet cards with expand-to-full-chunk toggle.
Upload.jsx	react-dropzone multi-file queue. Shows per-file upload status, supports force re-index checkbox.
MeetingUpload.jsx	Two-phase upload: calls /ingest-meeting first (fast), then /summarize-meeting (slow LLM). Shows "Uploading..." → "Summarizing..." step indicators. Displays meeting summary, source chunks, TTT entry ID, and TTT push errors.
Sources.jsx	Table of all indexed sources with chunk counts. Supports per-row delete with confirmation.

API layer (api.js)

```
ingestMeeting({ file, force })           // POST /ingest-meeting (multipart)
summarizeMeeting({ filename, ... })      // POST /summarize-meeting (JSON)
uploadFile(file, force)                  // POST /upload (multipart)
searchDocs({ query, top_k, ... })        // POST /search (JSON)
chatWithKB({ question, history, ... })   // POST /chat (JSON)
getSources()                             // GET /sources
deleteSource(source)                     // DELETE /sources?source=...
```

throwApiError(res) is a shared helper that parses FastAPI error responses — tries res.json().detail first, falls back to res.text(). This ensures rate-limit and validation messages surface as clean strings rather than raw JSON.

13. Deployment Model

Local

Service	How to start
pgvector	<code>docker compose up -d</code>
Ollama	<code>ollama serve</code>
FastAPI	<code>uvicorn api:app --reload</code> (from <code>backend/</code>)
Frontend	<code>npm run dev</code> (from <code>frontend/</code>)

Set `LLM_PROVIDER=ollama` and `EMBED_PROVIDER=ollama` in `.env`. No external API keys needed.

Cloud (Render + Neon)

Component	Provider
Backend API	Render web service (Python runtime)
Frontend	Render static site
Vector DB	Neon managed PostgreSQL + pgvector extension
Embeddings	Nomic Atlas API (<code>EMBED_PROVIDER=nomic</code>)
LLM	Groq via OpenAI-spec provider (<code>LLM_PROVIDER=openai</code> , <code>OPENAI_BASE_URL=https://api.groq.com/openai/v1</code>)
TTT DB	Neon managed PostgreSQL (separate database)

`render.yaml` defines the backend service with all environment variable keys. Secrets (`DATABASE_URL` , `OPENAI_API_KEY` , `NOMIC_API_KEY` , etc.) are set as `sync: false` and managed through the Render dashboard.

Render timeout constraint: Render free-tier enforces a 30-second response timeout. This is why the meeting pipeline is split into two requests — `/ingest-meeting` returns in seconds, and `/summarize-meeting` (which calls the LLM) is issued as a separate follow-up request by the frontend.

Environment variable precedence

Render dashboard env vars (highest – overrides `render.yaml`)

↓

`render.yaml` envVars values

↓

.env file (local only)

↓

config.py defaults (lowest)